Reviewer Pack — Minimal Rank of Nonnegative Tensor Factorizations vs Monoton…

Entry b6fcb6cbcf7b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 18:31:08 UTC

Minimal Rank of Nonnegative Tensor Factorizations vs Monotone Circuit Size for k-CLIQUE

Entry ID: `b6fcb6cbcf7b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-24 18:31:08 UTC

1. Conjecture

Field A (mathematical branch): Nonnegative Tensor Theory **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity for k-CLIQUE

Statement:

> ['For every instance of the k-CLIQUE problem with n variables, there exists a nonnegative tensor T such that its minimal rank is O(n^{1/2}), and any monotone circuit computing k-CLIQUE with size S must have S ≥ c * sqrt(n) for some constant c.', 'If a monotone circuit with size S computes k-CLIQUE, then the minimal rank of the nonnegative tensor representing it is at least S^2 / n.', 'For any constant ε > 0, there exists an instance of the k-CLIQUE problem such that the smallest possible value of c for which the above inequality holds is at least 1/ε.']

Rationale (proposer's reasoning):

> ['Nonnegative tensor factorizations can represent complex data with low-rank structures. It has been used in other areas of computer science, but its application to monotone circuits is novel.', 'The conjecture leverages the connection between nonnegative tensors and matrix rank, which is closely related to circuit complexity. If true, it would provide a new way to understand the inherent difficulty of k-CLIQUE.', 'Monotone circuits are often used as a model for proving lower bounds on classical computation problems. By connecting nonnegative tensor factorizations to monotone circuits, the conjecture may uncover new structural insights into the complexity of k-CLIQUE.']

Taxonomy category: `MONOTONE_CLIQUE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `54c7a554f3e6676d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for all n in {10, 20, ..., 40}, the minimal rank of the nonnegative tensor is O(n^{1/2}) and the size S of any monotone circuit computing k-CLIQUE

satisfies S ≥ c * sqrt(n), with a p-value from a significance test on 30 random seeds greater than 0.95; otherwise it is falsified.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	UNCERTAIN	1.00	UNCERTAIN	HITS
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `tensor theory minimal rank nonnegative AND monotone circuit complexity k-CLIQUE` - `nonnegative tensor factorization O(n^{1/2}) AND k-CLIQUE monotone circuit size` - `monotone circuit size S^2/n lower bound nonnegative tensor minimal rank`

Top relevant hits considered: - [http://arxiv.org/abs/1601.05351v3] Semialgebraic Geometry of Nonnegative Tensor Rank - [http://arxiv.org/abs/2510.02583v2] The Log-Rank Conjecture: New Equivalent Formulations - [http://arxiv.org/abs/hep-th/9707234v2] Variational Approach to Quantum Field Theory: Gaussian Approximation and the Perturbative Expansion around It - [http://arxiv.org/abs/0901.0512v4] Expected Performance of the ATLAS Experiment - Detector, Trigger and Physics - [http://arxiv.org/abs/2311.04204v3] Sharp Thresholds Imply Circuit Lower Bounds: from random 2-SAT to Planted Clique - [http://arxiv.org/abs/2303.17007v2] Impact of cross-section uncertainties on supernova neutrino spectral parameter fitting in the Deep Underground Neutrino - [http://arxiv.org/abs/1806.02734v3] Spectral lower bounds for the orthogonal and projective ranks of a graph - [http://arxiv.org/abs/2411.02936v3] Conditional Complexity Hardness: Monotone Circuit Size, Matrix Rigidity, and Tensor Rank

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_clique_instance(n):
        edges = []
        for i in range(n):
            for j in range(i + 1, n):
                if random.random() < 0.5:
                    edges.append((i, j))
        return edges

    def tensor_rank(edges):
```

```
        n = len(edges) + 1
        T = [[0] * n for _ in range(n)]
        for u, v in edges:
            T[u][v] = T[v][u] = 1
        rank = 0
        while True:
            found = False
            for i in range(n):
                if any(T[i][j] != 0 for j in range(i + 1, n)):
                    pivot = next(j for j in range(i + 1, n) if T[i][j] != 0)
                    for k in range(n):
                        if k != i:
                            factor = T[k][pivot] / T[i][pivot]
                            for j in range(n):
                                T[k][j] -= factor * T[i][j]
                    found = True
            if not found:
                break
            rank += 1
        return rank

    def monotone_circuit_size(edges):
        n = len(edges) + 1
        circuit_size = 0
        for u, v in edges:
            circuit_size += 2  # Each edge requires at least two gates (AND and OR)
        return circuit_size

    results = []
    for _ in range(30):  # Test with 30 random instances
        n = random.choice([10, 20, 30, 40])
        edges = generate_clique_instance(n)
        T_rank = tensor_rank(edges)
        circuit_size = monotone_circuit_size(edges)
        results.append((T_rank, circuit_size))

    mean_T_rank = sum(T_rank for T_rank, _ in results) / len(results)
    mean_circuit_size = sum(circuit_size for _, circuit_size in results) / len(results)
    support_fraction = sum(1 for T_rank, circuit_size in results if T_rank <= math.sqrt(n) and cir

    conjecture_holds = support_fraction > 0.95
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "minimal_rank_vs_circuit_size",
        "metric_value": mean_T_rank,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 2
    results = []
```

```python
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) /
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means it did not provide evidence to support or falsify the conjecture. | next: Re-run the test with increased time limits and ensure that it completes without crashing. If the test passes, re-evaluate the results against the pre-registered criteria.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13781
2	preregistration	ollama_remote	glm4:latest	0	0	6210
3	novelty	ollama_remote	glm4:latest	0	0	4827
4	novelty	ollama_remote	glm4:latest	0	0	6346
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16815
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8725
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10035
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10693

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	judge	ollama_remote	glm4:latest	0	0	8387

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 85819 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/b6fcb6cbcf7b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b6fcb6cbcf7b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b6fcb6cbcf7b.tar.gz` (if generated)